

SIMATS ENGINEERING

CO3-AT1-DEBUGGING

COURSE CODE: CSA0904

NAME: Rose Mystica. J

COURSE NAME: Programming in JAVA

REG NO: 192421410

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

ROSE MYSTICA J
192421410RA

CO3-AT1-Debugging

← Back

QUESTIONS**Q1** Basic Thread Creation and Start
25 marks**Q2** Try-Catch-Finally Basics
25 marks**Q3** wait() and notify() Coordination
25 marks**Q4** Runnable vs Callable Confusion
25 marks

4/4 answered

Q1 Basic Thread Creation and Start 25 marks

```
public class ThreadBasics {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new MyTask());  
        t1.run();  
        MyTask task = new MyTask();  
        task.start();  
  
        Thread t2 = new Thread(task);  
        t2.start();  
        t2.start();  
  
        System.out.println(t1.getName());  
  
        try {  
            t2.join();  
        } catch (InterruptedException e) {  
        }  
    }  
}
```

Your Code

```
1 public class ThreadBasics {  
2     public static void main(String[] args) {  
3         Thread t1 = new Thread(new MyTask());  
4         t1.start();  
5         MyTask task = new MyTask();  
6         task.run();  
7         Thread t2 = new Thread(task);  
8         t2.start();  
9         System.out.println(t1.getName());  
10        try {  
11            t2.join();  
12        } catch (InterruptedException e) {  
13        }  
14    }  
15    static class MyTask implements Runnable {  
16        public void run() {  
17            System.out.println("Task running on: " + Thread.currentThread().getName());  
18        }  
19    }  
20 }
```

Run **Save**

Input

stdin input

Output

Task running on: Thread-0
Task running on: main
Thread-0
Task running on: Thread-1

Q2 Try-Catch-Finally Basics

25 marks

```
public class TryCatchFinallyDemo {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};

        try {
            System.out.println(numbers[5]); // ArrayIndexOutOfBoundsException expected here
        } catch (NullPointerException e) { // Bug 1: wrong exception type caught, real exception not handled
            System.out.println("Caught null pointer");
        }

        try {
            int value = Integer.parseInt("abc");
        } catch (Exception e) {
        }

        String str = null;
        try {
            System.out.println(str.length());
        } catch (NullPointerException e) {
```

**SIMATS Online Programming Lab**

JAVA PROGRAMMING

ROSE MYSTICA J
192421410RA

Your Code

```
1 public class TryCatchFinallyDemo {
2     public static void main(String[] args) {
3         int[] numbers = {1, 2, 3};
4         try {
5             System.out.println(numbers[5]);
6         } catch (ArrayIndexOutOfBoundsException e) {
7             System.out.println("Caught array error");
8         }
9         try {
10            int value = Integer.parseInt("abc");
11        } catch (NumberFormatException e) {
12        }
13        String str = null;
14        try {
15            System.out.println(str.length());
16        } catch (NullPointerException e) {
17            return;
18        } finally {
19            System.out.println("Cleanup");
20        }
21        try {
22            riskyMethod();
23        } catch (java.io.IOException e) {
24            e.printStackTrace();
25        }
26    }
27    static void riskyMethod() throws java.io.IOException {
```

Input

stdin input

Output

Caught array error
Cleanup

Q3 wait() and notify() Coordination

25 marks

```
public class WaitNotifyDemo {
    private boolean dataReady = false;
    private final Object lock = new Object();

    public void produce() {
        synchronized (lock) {
            dataReady = true;
            System.out.println("Data produced");
            lock.notify();
        }
    }

    public void consume() {
        if (!dataReady) {
            try {
                lock.wait();
            } catch (InterruptedException e) {
            }
        }
    }
}
```

**SIMATS Online Programming Lab**

JAVA PROGRAMMING

ROSE MYSTICA J
192421410RA**Your Code**

```
1 public class WaitNotifyDemo {
2     private boolean dataReady = false;
3     private final Object lock = new Object();
4     public void produce() {
5         synchronized (lock) {
6             dataReady = true;
7             System.out.println("Data produced");
8             lock.notify();
9         }
10    }
11    public void consume() {
12        if (!dataReady) {
13            try {
14                synchronized (lock) {
15                    lock.wait();
16                }
17            } catch (InterruptedException e) {
18            }
19        }
20        System.out.println("Data consumed");
21    }
22    public void consumeCorrectly() {
23        synchronized (lock) {
24            if (!dataReady) {
25                try {
26                    lock.wait();
```

Input

stdin input

Output

Data produced

Q4 Runnable vs Callable Confusion

25 marks

```
import java.util.concurrent.*;

public class RunnableCallableDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        Callable<Integer> callableTask = () -> {
            return 10 / 0;
        }

        Future<Integer> future = executor.submit(callableTask);
        int result = future.get(); caller context
        System.out.println(result);

        Runnable r = () -> {
            return 5;
        };
        executor.submit(r);

        executor.shutdown();
    }
}
```

Your Code

```
1 import java.util.concurrent.*;
2 public class RunnableCallableDemo {
3     public static void main(String[] args) throws Exception {
4         ExecutorService executor =
5             Executors.newFixedThreadPool(2);
6         Callable<Integer> callableTask = () -> {
7             return 10 / 0;
8         };
9         Future<Integer> future = executor.submit(callableTask);
10        try {
11            int result = future.get();
12            System.out.println(result);
13        } catch (ExecutionException e) {
14            System.out.println(e.getCause());
15        }
16        Runnable r = () -> {
17            System.out.println("Runnable executed");
18        };
19        executor.submit(r);
20        executor.shutdown();
21    }
22 }
```

Input

stdin input

Output

```
java.lang.ArithmeticException: / by
zero
Runnable executed
```